The Ultimate CFT Grimoire

Part 1: 1D CFT & Part 2: 2D CFT (Properties and Pairs)

Prerequisite: The 1D CFT Framework

Your assignment provided a 2D CFT class. To test 1D properties, we must define a 1D equivalent. Add this to your script. It uses `np.trapezoid` and avoids all built-in FFTs.

```
import numpy as np

def calculate_mse(arr1, arr2):
    return np.mean(np.abs(arr1 - arr2)**2)

class CFT1D:
    """Computes the 1D Continuous Fourier Transform numerically."""
    def __init__(self, t, x_t):
        self.t = t
        self.x_t = x_t
        self.dt = t[1] - t[0]
        # Define frequency axis (Nyquist range)
        self.f = np.linspace(-1/(2*self.dt), 1/(2*self.dt), len(t))

    def compute_cft(self):
        X_f = np.zeros(len(self.f), dtype=complex)
        for i, freq in enumerate(self.f):
            integrand = self.x_t * np.exp(-1j * 2 * np.pi * freq * self.t)
            X_f[i] = np.trapezoid(integrand, self.t)
        return X_f

# Setup a base signal for 1D testing: A Gaussian pulse
t = np.linspace(-5, 5, 2000)
dt = t[1] - t[0]
x_base = np.exp(-t**2)
cft_base = CFT1D(t, x_base)
X_base = cft_base.compute_cft()
f = cft_base.f
```

Part 1: 1D Continuous Fourier Transform

1 1D CFT Properties

1.1 1. Time Shifting & Frequency Shifting (Modulation)

Problem Statement: Consider a radar pulse $x(t)$. The pulse is delayed by $t_0 = 1.5$ seconds due to travel time, and modulated by a carrier frequency $f_0 = 2.0$ Hz for transmission. Verify the Time and Frequency shift properties.

Theory: Time Shift: $x(t - t_0) \leftrightarrow X(f)e^{-j2\pi f t_0}$. (Delay in time = linear phase shift in frequency). Freq Shift: $x(t)e^{j2\pi f_0 t} \leftrightarrow X(f - f_0)$. (Multiplying by a wave in time = shifting the spectrum).

```
# --- Time Shift ---
t0 = 1.5
# Interpolate to shift the continuous signal
x_tshift = np.interp(t - t0, t, x_base, left=0, right=0)
X_tshift_computed = CFT1D(t, x_tshift).compute_cft()
X_tshift_theo = X_base * np.exp(-1j * 2 * np.pi * f * t0)
print(f"Time Shift MSE: {calculate_mse(X_tshift_computed, X_tshift_theo)}")

# --- Frequency Shift ---
f0 = 2.0
x_fshift = x_base * np.exp(1j * 2 * np.pi * f0 * t)
X_fshift_computed = CFT1D(t, x_fshift).compute_cft()
# Shift the theoretical array by finding the index offset
df = f[1] - f[0]
shift_idx = int(f0 / df)
X_fshift_theo = np.roll(X_base, shift_idx)
print(f"Freq Shift MSE: {calculate_mse(X_fshift_computed, X_fshift_theo)}")
```

1.2 2. Time & Frequency Scaling

Problem Statement: A recorded audio signal $x(t)$ is played back at double speed ($a = 2$). Verify the scaling property.

Theory: $x(at) \leftrightarrow \frac{1}{|a|}X\left(\frac{f}{a}\right)$. Compressing a signal in time ($a > 1$) causes it to change faster, which *expands* its frequency spectrum and reduces its overall amplitude density.

```
a = 2.0
x_scaled = np.interp(a * t, t, x_base, left=0, right=0)
X_scaled_computed = CFT1D(t, x_scaled).compute_cft()

# Theoretical: 1/|a| * X(f/a). We interpolate the frequency array!
X_scaled_theo = (1/np.abs(a)) * np.interp(f/a, f, X_base.real) + \
           1j * (1/np.abs(a)) * np.interp(f/a, f, X_base.imag)

print(f"Time Scaling MSE: {calculate_mse(X_scaled_computed, X_scaled_theo)}")
```

1.3 3. Duality Property

Problem Statement: Verify the Duality property. If $x(t) \leftrightarrow X(f)$, then $X(t) \leftrightarrow x(-f)$.

Theory: The Fourier Transform and its Inverse are almost identical integrals, differing only by a minus sign in the exponent. Therefore, if you plug a frequency-domain shape into the time-domain, its CFT will be the time-domain shape, reversed! (Note: In the ω table, duality has a 2π factor. Using f, the 2π vanishes perfectly: $X(t) \leftrightarrow x(-f)$).

```
# Let's use the computed X_base as our new TIME domain signal
# X_base is defined over the frequency grid `f`. We need it over `t`.
```

```
# For simplicity, we interpolate X_base onto the `t` grid.
X_as_time_real = np.interp(t, f, X_base.real)
X_as_time_imag = np.interp(t, f, X_base.imag)
X_as_time = X_as_time_real + 1j * X_as_time_imag

# Compute CFT of this new signal
X_dual_computed = CFT1D(t, X_as_time).compute_cft()

# Theoretical: x(-f). We interpolate x_base onto the reversed -f grid.
x_dual_theo = np.interp(-f, t, x_base, left=0, right=0)

print(f"Duality MSE: {calculate_mse(X_dual_computed, x_dual_theo)}")
```

1.4 4. Differentiation in Time & Frequency

Problem Statement: Verify that taking the derivative of $x(t)$ corresponds to multiplying by $j2\pi f$, and multiplying $x(t)$ by t corresponds to differentiating in frequency.

Theory: Time Deriv: $\frac{d}{dt}x(t) \leftrightarrow j2\pi f X(f)$. (Table uses $j\omega$). Freq Deriv: $tx(t) \leftrightarrow \frac{j}{2\pi}\frac{d}{df}X(f)$. (Table uses $j\frac{d}{d\omega}$).

```
# --- Differentiation in Time ---
x_diff_t = np.gradient(x_base, dt)
X_diff_t_computed = CFT1D(t, x_diff_t).compute_cft()
X_diff_t_theo = 1j * 2 * np.pi * f * X_base
print(f"Time Deriv MSE: {calculate_mse(X_diff_t_computed, X_diff_t_theo)}")

# --- Differentiation in Frequency ---
x_t_mult = t * x_base
X_t_mult_computed = CFT1D(t, x_t_mult).compute_cft()

# Numerical derivative of the spectrum
df = f[1] - f[0]
dX_df = np.gradient(X_base, df)
X_t_mult_theo = (1j / (2 * np.pi)) * dX_df
print(f"Freq Deriv MSE: {calculate_mse(X_t_mult_computed, X_t_mult_theo)}")
```

1.5 5. Convolution & Multiplication

Problem Statement: Verify that convolution in time equals multiplication in frequency, and vice versa.

Theory: $x(t) * y(t) \leftrightarrow X(f)Y(f)$. $x(t)y(t) \leftrightarrow X(f) * Y(f)$. (Note: Table shows $\frac{1}{2\pi}$ for ω. With f, there is NO scaling factor!).

```
y_base = np.exp(-2 * t**2)
Y_base = CFT1D(t, y_base).compute_cft()

# --- Convolution in Time ---
# np.convolve gives a larger array. We use mode='same' and multiply by dt
# to approximate the continuous integral.
x_conv_y = np.convolve(x_base, y_base, mode='same') * dt
X_conv_computed = CFT1D(t, x_conv_y).compute_cft()
X_conv_theo = X_base * Y_base
print(f"Time Convolution MSE: {calculate_mse(X_conv_computed, X_conv_theo)}")

# --- Multiplication in Time ---
x_mult_y = x_base * y_base
X_mult_computed = CFT1D(t, x_mult_y).compute_cft()
# Convolution in frequency domain (multiply by df)
X_mult_theo = np.convolve(X_base, Y_base, mode='same') * df
print(f"Time Multiplication MSE: {calculate_mse(X_mult_computed, X_mult_theo)}")
```

1.6 6. Parseval's Relation

Theory: $\int |x(t)|^2 dt = \int |X(f)|^2 df$. Total energy is conserved across domains.

```
energy_t = np.trapezoid(np.abs(x_base)**2, t)
energy_f = np.trapezoid(np.abs(X_base)**2, f)
print(f"Parseval - Energy Time: {energy_t:.4f}, Energy Freq: {energy_f:.4f}")
```

---

2 1D Transform Pairs Verification

Note on Deltas (δ): Numerically, a delta function cannot be perfectly represented. We approximate it, or we verify pairs that don't contain deltas.

2.1 Pair 1: Rectangular Pulse $\leftrightarrow$ Sinc

Theory: $rect(t/T_1) \leftrightarrow T_1 \text{sinc}(T_1 f)$. (Where $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$).

```
T1 = 2.0
x_rect = np.where(np.abs(t) <= T1/2, 1.0, 0.0)
X_rect_computed = CFT1D(t, x_rect).compute_cft()

# np.sinc in python is defined as sin(pi*x)/(pi*x), which perfectly matches our f-
    domain!
X_rect_theo = T1 * np.sinc(T1 * f)
print(f"Rect <-> Sinc MSE: {calculate_mse(X_rect_computed, X_rect_theo)}")
```

2.2 Pair 2: Two-Sided Exponential $\leftrightarrow$ Lorentzian

Theory: $e^{-a|t|} \leftrightarrow \frac{2a}{a^2+(2\pi f)^2}$.

```
a = 1.5
x_exp = np.exp(-a * np.abs(t))
X_exp_computed = CFT1D(t, x_exp).compute_cft()
X_exp_theo = (2 * a) / (a**2 + (2 * np.pi * f)**2)
print(f"Exp <-> Lorentzian MSE: {calculate_mse(X_exp_computed, X_exp_theo)}")
```

2.3 Pair 3: Triangle $\leftrightarrow$ Sinc^2

Theory: $tri(t/T_1) \leftrightarrow T_1 \text{sinc}^2(T_1 f)$.

```
T1 = 2.0
x_tri = np.where(np.abs(t) <= T1, 1.0 - np.abs(t)/T1, 0.0)
X_tri_computed = CFT1D(t, x_tri).compute_cft()
X_tri_theo = T1 * (np.sinc(T1 * f))**2
print(f"Tri <-> Sinc^2 MSE: {calculate_mse(X_tri_computed, X_tri_theo)}")
```

Part 2: 2D Continuous Fourier Transform

We now apply these concepts to the 2D spatial domain using your assignment's `CFT2D` class.

```
import copy
from imageio.v2 import imread

def get_complex_cft(cft_obj):
    real, imag = cft_obj.compute_cft()
    return real + 1j * imag

def get_uv_grids(cft_obj):
    return np.meshgrid(cft_obj.u, cft_obj.v)

# Assume ContinuousImage and CFT2D are defined as per your assignment
img = ContinuousImage("pikachu.png")
cft_orig = CFT2D(img)
F_orig = get_complex_cft(cft_orig)
U, V = get_uv_grids(cft_orig)
```

3 2D CFT Properties

3.1 1. 2D Spatial Shifting

Theory: $I(x - x_0, y - y_0) \leftrightarrow F(u, v)e^{-j2\pi(ux_0+vy_0)}$.

```
x0, y0 = 0.2, -0.1
dx, dy = img.x[1] - img.x[0], img.y[1] - img.y[0]
img_shifted = copy.deepcopy(img)
img_shifted.image = np.roll(img.image, int(x0/dx), axis=1)
img_shifted.image = np.roll(img_shifted.image, int(y0/dy), axis=0)

F_shifted_computed = get_complex_cft(CFT2D(img_shifted))
F_shifted_theo = F_orig * np.exp(-1j * 2 * np.pi * (U * x0 + V * y0))
print(f"2D Spatial Shift MSE: {calculate_mse(F_shifted_computed, F_shifted_theo)}")
```

3.2 2. 2D Modulation (Frequency Shift)

Theory: $I(x, y)e^{j2\pi(u_0x+v_0y)} \leftrightarrow F(u - u_0, v - v_0)$.

```
u0, v0 = 5.0, 10.0
X, Y = np.meshgrid(img.x, img.y)
mod_arr = img.image * np.exp(1j * 2 * np.pi * (u0 * X + v0 * Y))

# Compute CFT of real and imag parts separately due to class constraints
img_r, img_i = copy.deepcopy(img), copy.deepcopy(img)
img_r.image, img_i.image = mod_arr.real, mod_arr.imag
F_mod_computed = get_complex_cft(CFT2D(img_r)) + 1j * get_complex_cft(CFT2D(img_i))

du, dv = cft_orig.u[1] - cft_orig.u[0], cft_orig.v[1] - cft_orig.v[0]
F_mod_theo = np.roll(F_orig, int(u0/du), axis=1)
F_mod_theo = np.roll(F_mod_theo, int(v0/dv), axis=0)
print(f"2D Modulation MSE: {calculate_mse(F_mod_computed, F_mod_theo)}")
```

3.3 3. 2D Differentiation (The Laplacian)

Theory: $\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \leftrightarrow -4\pi^2(u^2 + v^2)F(u, v)$.

```
I_xx = np.gradient(np.gradient(img.image, dx, axis=1), dx, axis=1)
I_yy = np.gradient(np.gradient(img.image, dy, axis=0), dy, axis=0)
```

```
img_lap = copy.deepcopy(img)
img_lap.image = I_xx + I_yy

F_lap_computed = get_complex_cft(CFT2D(img_lap))
F_lap_theo = -4 * (np.pi**2) * (U**2 + V**2) * F_orig
print(f"2D Laplacian MSE: {calculate_mse(F_lap_computed, F_lap_theo)}")
```

3.4 4. 2D Parseval's Theorem

Theory: $\iint |I(x,y)|^2 dxdy = \iint |F(u,v)|^2 dudv$.

```
energy_spatial = np.trapezoid(np.trapezoid(np.abs(img.image)**2, img.x, axis=1),
    img.y)
energy_freq = np.trapezoid(np.trapezoid(np.abs(F_orig)**2, cft_orig.u, axis=1),
    cft_orig.v)
print(f"2D Parseval - Spatial: {energy_spatial:.4f}, Freq: {energy_freq:.4f}")
```

---

4 2D Transform Pairs Verification

4.1 Pair 1: 2D Rectangular Aperture $\leftrightarrow$ 2D Sinc

Theory: A 2D square pulse $rect(x/W)rect(y/W)$ transforms to $W^2\text{sinc}(Wu)\text{sinc}(Wv)$.

```
W = 0.5
rect_2d = np.where((np.abs(X) <= W/2) & (np.abs(Y) <= W/2), 1.0, 0.0)
img_rect = copy.deepcopy(img)
img_rect.image = rect_2d
F_rect_computed = get_complex_cft(CFT2D(img_rect))

F_rect_theo = (W**2) * np.sinc(W * U) * np.sinc(W * V)
print(f"2D Rect <-> Sinc MSE: {calculate_mse(F_rect_computed, F_rect_theo)}")
```

4.2 Pair 2: 2D Gaussian $\leftrightarrow$ 2D Gaussian

Theory: $e^{-\pi(x^2+y^2)} \leftrightarrow e^{-\pi(u^2+v^2)}$. The 2D Gaussian is the eigenfunction of the 2D CFT.

```
gauss_2d = np.exp(-np.pi * (X**2 + Y**2))
img_gauss = copy.deepcopy(img)
img_gauss.image = gauss_2d
F_gauss_computed = get_complex_cft(CFT2D(img_gauss))

F_gauss_theo = np.exp(-np.pi * (U**2 + V**2))
print(f"2D Gaussian <-> Gaussian MSE: {calculate_mse(F_gauss_computed, F_gauss_theo
    )}")
```